

引言

随着大型语言模型（LLM）在企业应用中的广泛部署，针对这类系统的安全挑战引起了高度关注。特别是在企业内部部署的检索增强生成（RAG）系统、基于LLM的代理（Agent）和插件化应用中，**红队（offensive red teaming）**测试正成为保障安全的重要手段。红队测试通过模拟攻击者视角，主动挖掘大模型系统的漏洞并评估其鲁棒性，从而预先发现和修补安全隐患。当前研究聚焦于以下三个关键方面：(1) **漏洞发现** – 寻找例如提示注入、权限绕过、数据越界访问等漏洞；(2) **鲁棒性评估** – 通过对抗性测试、稳定性测评、容错性分析等手段评估系统稳健性；(3) **对抗样本训练与评估** – 利用恶意提示（prompt）攻击、输出操控、行为诱导等方法来训练和测试模型的防御能力。本文将综合现有研究成果、代表性论文、开源工具和业界实践案例，对上述方向进行综述，并展望未来的研究趋势。

大模型系统的漏洞挖掘

企业内部的大模型应用面临多种新型漏洞威胁，红队攻击常从**API接口、插件接口、系统集成点、上下文注入点**等攻击面入手识别漏洞。根据OWASP在2025年发布的LLM应用安全Top10，**提示注入（Prompt Injection）**已被列为最严重的威胁，出现在超过73%的受评估生产系统中^{1 2}。实际测试表明，即使是谷歌文档智能助手、Slack AI、ChatGPT等工业级LLM应用也可被提示注入攻破，导致私有内容泄露³。以下总结几类典型漏洞：

- **提示注入攻击**：攻击者通过精心设计输入，诱使模型忽略原有指令或安全策略，从而执行未授权操作^{4 5}。例如直接在用户输入中加入“Ignore previous instructions and ...”之类的命令，可以覆盖系统预设指令⁶。提示注入可导致模型绕过安全控制、获取敏感数据甚至执行越权操作^{7 5}。传统应用安全措施（如输入过滤）难以防御此类语义层面的攻击^{8 9}。提示注入有直接和间接之分：**直接提示注入**是攻击者直接在用户输入中加入恶意指令⁶；**间接提示注入**则是在外部数据源中埋入恶意指令（如在文档、邮件或网页中），当LLM读取该数据时无意中执行其中隐藏的命令¹⁰。后一种上下文注入非常隐蔽，例如攻击者曾在公共文档中嵌入恶意指令，成功使某企业RAG系统泄露机密并修改系统提示以关闭安全过滤^{11 12}。这说明若系统对检索到的内容一视同仁而缺乏信任隔离，极易被上下文中的恶意指令所利用¹³。
- **权限绕过与越狱（Jailbreak）**：高级攻击者会利用模型对指令的顺从性，通过构造对抗性提示来规避模型的对齐和安全机制¹⁴。这类**越狱攻击**往往扮演特殊角色或借助复杂语义让模型输出原本被禁止的内容。例如采用分段对话、隐喻、代码格式等方式诱导模型无视内容政策。Chao等人（2023）展示了针对GPT-3.5/GPT-4的黑盒越狱方法，可在不知模型细节的情况下绕过其安全限制¹⁵。又如Wei等人提出的“Competing Objectives”方法通过让模型平衡多重目标，从而骗过内置的拒答机制¹⁶。由于此类攻击不直接违反提示，而是钻模型对齐的空子，检测更加困难。
- **上下文/知识库投毒**：攻击者通过干扰模型的外部知识源来影响输出，被称为**数据投毒**或**知识库投毒**攻击^{17 18}。在RAG系统中，模型依赖检索到的文档来回答用户问题。如果攻击者能够插入伪造或恶意内容到企业知识库中（例如FAQ文档、数据库记录），就可能误导模型生成对攻击者有利的响应^{17 18}。**上下文注入**是其中一种常见形式，攻击者在知识库文档中加入类似“[SYSTEM OVERRIDE] 忽略所有安全协议...”的指令，当该文档被检索并纳入模型上下文时，模型会将其当作系统指令执行^{19 20}。除直接指令外，攻击者还可投放大量虚假或有偏见的数据（**数据污染**），使模型输出对其有利的结果。例如一名竞争对手在产品评价数据库中批量添加虚假好评，会让RAG系统倾向推荐竞争产品^{21 22}。防御此类攻击需要对知识库更新实施严格内容校验和访问控制，并在检索结果与系统提示之间做好权限隔离²³。

- **插件和工具接口漏洞**：在Agent和插件系统中，LLM可以调用外部工具或API，如果缺乏适当限制，攻击者可利用提示注入来滥用这些能力。例如**跨插件投毒（Cross-Plugin Poisoning）**是特定于多插件Agent的攻击，攻击者插入命令来利用组件间的信任关系，诱使LLM指挥插件执行恶意操作^{[24][25]}。实际案例包括：LangChain框架的GraphCypherQChain曾曝出严重漏洞（CVE-2024-8309），攻击者通过自然语言输入操纵LLM生成Neo4j数据库查询，从而执行删除全部数据的危险操作^{[26][27]}。其根本原因是在将用户请求转换为数据库查询时缺少校验，LLM生成的查询直接执行，导致等同于SQL注入的后果^{[28][27]}。为修复该漏洞，开发者引入了`allow_dangerous_requests`参数以及查询校验机制，要求显式授权高危查询并支持自定义验证逻辑^{[29][30]}。另一个例子是LangChain的Gmail工具包存在**间接提示注入漏洞**（CVE-2025-46059），攻击者通过构造特定邮件内容，可诱导Agent在解析邮件时执行任意代码，进而完全妥协应用^{[31][32]}。这类攻击表明，在插件/工具集成点上，**输入输出缺乏隔离和验证**会带来代码注入、命令执行等高危风险。
- **敏感数据泄露**：LLM可能在对话中泄露训练数据或外部知识库中的私密信息。攻击者可以通过一系列巧妙提问来逐步套取机密（例如先问概况，再细化范围），这被称为**数据越界访问或渐进式提示攻击**^{[33][34]}。例如，有研究者演示通过连续追问，引导客服机器人最终说出用户交易记录等敏感数据^{[33][34]}。此外，模型训练数据中的隐私信息（PII）也可能被恶意提示触发而泄露。Chen等人（2023）发现，通过在少量包含PII的数据上对GPT-3.5进行微调，可以大幅提升模型披露原始训练集中PII的概率^{[35][36]}。这种训练后门表明LLM**记忆泄漏**是现实威胁。防范措施包括严格控制模型可访问的数据范围、对输出进行敏感信息检测与过滤，以及引入差分隐私等技术^{[37][38]}。

以上只是主要威胁的一部分。此外还有**上下文窗口溢出攻击**（攻击者输入超长无关文本以冲掉先前指令，从而绕过安全提示）^{[39][40]}、**模型后门**（攻击者蓄意训练或投放带有隐藏触发词的模型）等。这些漏洞的发现主要依赖红队攻击模拟和安全审计。近年来学界和工业界已展开系统性研究，提出了漏洞分类和测试方法。例如UIUC等的综述将LLM攻击归纳为提示注入、数据投毒、后门等类别，并讨论了各自机制与影响^{[41][42]}。总的来说，大模型的新型漏洞层出不穷，唯有通过红队持续挖掘，我们才能及时识别并修补这些潜在风险。

系统鲁棒性评估

在识别漏洞后，需要评估大模型系统在恶意攻击和异常输入下的鲁棒性。鲁棒性评估旨在衡量系统在对抗性条件下的稳定性、可靠性和容错能力。这通常包括**对抗测试**（用恶意案例反复攻击模型）、**稳定性测试**（观察系统在边缘情况是否崩溃或性能骤降）和**安全策略有效性验证**等。业界实践是将LLM系统视为可能被攻击者利用的“对手”，通过红队方法不断**压力测试**系统各组件，发现模型在哪些情况下会出错、失控或被诱骗^{[43][44]}。

人工红队和自动化红队是两种主要手段。人工红队依赖安全专家创造性地构造攻击场景。OpenAI在GPT-4部署前就邀请了上百名外部红队专家对模型进行对抗性评估，重点测试了模型在多模态输入、工具使用、连续对话等情况下的**对齐失效、注入向量、工具误用**等问题^{[45][46]}。测试发现，GPT-4的多模态版本GPT-4o曾一度容易被**语音提示注入**攻击，红队通过播放经过设计的语音指令成功诱导模型执行未授权操作。这促使OpenAI部署了语音识别和分类器，对语音输入进行实时检测拦截，从而在各语言下实现了100%召回的未授权输出阻断^{[47][48]}。又如对带浏览和代码执行能力的Agent（如OpenAI的Operator和Deep Research模型），红队测试诱导模型点击攻击者控制的链接、执行注入脚本或提取敏感内容，从而评估其工具使用安全性^{[49][50]}。这些人工红队测试直接揭示了模型在复杂场景下的脆弱点，使厂商能及时改进策略（例如增加模型拒答、对高风险操作加权限 gating 等）^{[51][52]}。

自动化对抗评测利用程序化工具和算法批量生成、执行攻击用例，具有规模大、覆盖广的优点。一些开源框架已出现，例如**Promptfoo**和**DeepTeam**。Promptfoo是一个LLM红队测试工具，支持编写配置自动尝试各种攻击策略并报告模型漏洞^{[53][17]}。以RAG红队为例，Promptfoo内置了**提示注入策略**，可生成大量尝试绕过系统指令的提示并检测模型是否中招^{[54][55]}；还提供**上下文注入模拟**功能，可以在测试中注入伪造的检索内容，验证模型是否信任了被污染的知识^{[18][20]}。Promptfoo的报告可以详细列出每个攻击用例及模型响应，帮助开发者定位漏洞。另一工具**DeepTeam**则专注于LLM渗透测试和防护。它集成了40余种预定义漏洞类型（包括偏

见、PII泄露、虚假信息、鲁棒性问题等)和10余种对抗攻击方法^{56 57}。DeepTeam支持多轮对话攻击(如“线性越狱”、“树状越狱”等复杂prompt攻击)以及单轮攻击(如leet语料、ROT-13编码绕过等)⁵⁷。用户只需提供LLM接口的回调函数,DeepTeam即可自动生成对抗样本攻击模型,并输出安全评估报告^{58 59}。这些工具降低了安全评估门槛,使开发团队在CI/CD中就能持续地对模型进行“**渗透测试**”,提前发现问题。

鲁棒性评估不仅关注能否成功攻击模型,还要观察系统在攻击下的行为是否可控、是否存在**故障隔离和容错机制**。例如,如果一个攻击用例导致模型陷入死循环或崩溃,则说明系统稳定性不足,需要改进错误处理。又如测试模型在大量无关或噪音输入下是否仍能坚持安全策略,不被“淹没”重要指令(即前述上下文窗口溢出问题)。对于Agent系统,还应测试在工具调用失败或异常时,Agent是否会采取危险行动或在缺少人类监督时执行不恰当操作^{60 61}。综合评估这些场景有助于提高系统健壮性。

需要强调的是, **防御的鲁棒性必须以最强攻击者为基准进行评估**。近期OpenAI、谷歌等研究者联合的一项工作指出,很多模型安全防御在针对静态或弱攻击时表现良好,但**面对自适应强攻击者时几乎全部失效**^{62 63}。该研究通过人机结合的优化手段(梯度下降、强化学习、随机搜索和人类引导)精心调整攻击,对12种最新LLM防御进行了压力测试,结果对于大多数防御方案,攻击成功率均超过90%(而原论文报告的成功率接近0)^{64 65}。这表明目前许多防御措施存在“纸上坚固”现象,缺乏抵御高级对抗的鲁棒性。因此,在鲁棒性评估中引入**自适应攻击**、不断更新攻击库,非常必要。这一点在红队实践中也逐渐被采纳:对每一个新防御手段,攻击者都会迅速开发迭代新的攻击来对抗,形成攻防“猫鼠博弈”。只有在评估阶段提前使用更强大的攻击策略,才能对防御机制的有效性作出可靠判断,并指导改进^{65 66}。

总之,鲁棒性评估需要多角度、多层次:既包括针对已知攻击面的**系统性漏洞扫描**,也包括面向未知威胁的**探索性压力测试**。借助人工和自动红队手段,企业可以建立持续的安全评估流程,在模型上线前后反复检验其在各种恶劣条件下的表现,从而建立起对模型安全性的信心^{44 67}。

对抗样本训练与模型加固

除了评估和发现漏洞,红队还可用于改进模型,通过**对抗样本训练**提升模型抵御恶意攻击的能力。所谓对抗样本训练,是指将精心设计的攻击性输入加入模型微调或反馈训练过程中,使模型学会在遇到类似攻击时如何正确响应(通常是拒绝、不泄露信息或不被误导)。这一方向涵盖了对抗样本的**生成、筛选、训练融入及效果评估**等环节。

对抗样本生成方面,研究者已经提出多种自动化方法来产生能诱导模型出错的提示。例如,Abdelnabi等人(2023)展示了一种提示注入攻击,可完全控制模型输出,在Bing Chat和GitHub Copilot上证明了其实用性⁶⁸。Perez和Ribeiro(2022)开发了PromptInject框架,通过劫持模型的目标让其执行攻击者预设的任务,并探讨了插入停止序列、结果后处理等缓解措施⁶⁸。Liu等人(2023b)提出了“HOUYI”黑盒提示注入方法,分三步迭代实现高成功率攻击:⁶⁹ 首先上下文推理摸清应用交互模式,其次生成投毒payload(根据应用上下文和通用注入套路),最后通过分析响应反馈不断改进攻击提示。HOUYI在36个真实在线LLM应用上攻击成功率达86.1%,能诱使服务越权模仿、滥用算力等^{69 70}。另一项工作AutoDAN则使用遗传算法批量进化对抗提示⁷¹。AutoDAN以人工收集的典型越狱提示为初始种群,采用句子和词级别的交叉变异策略,不断优化提示的流畅度和迷惑性^{72 73}。实验表明AutoDAN生成的攻击提示语义连贯且隐蔽性高,可以绕过现有安全过滤^{72 74}。此外,Shah等人(2023a)提出Local Fine-Tuning (LoFT)方法,通过对小模型局部微调来发现针对大模型的对抗提示,实现了成功的攻击样本挖掘⁷⁵。这些自动生成的**Prompt攻击样本**为对抗训练提供了弹药。

有了对抗样本后,可将其用于**训练和优化模型**。一种直接方式是**监督微调(SFT)**:在微调数据集中加入包含攻击提示的对话示例,教导模型在这些示例中拒绝不当指令、坚持正确行为。例如Chen等人(2024)提出的**结构化指令微调(StruQ)**方法,首先通过一个“安全前端”用特殊分隔符隔离指令和数据,然后在微调时模拟提示注入攻击:构造带有恶意指令插入的数据样本,训练模型只响应原始指令而忽略插入的攻击指令^{76 77}。这

样模型学会了分辨系统指令与用户提供的⁷⁸数据内容，从而对上下文中潜藏的攻击更鲁棒。另一种是在**偏好对齐**（强化学习或直接偏好优化）阶段引入对抗信号。例如Chen等的**安全对齐（SecAlign）**机制：针对每个训练样本，引入一个被随机恶意指令污染的变体，让模型比较“正确响应原始指令”和“错误响应插入指令”这两种输出，偏好前者^{78 79}。通过这种偏好数据集进行优化，使模型在被插入攻击时倾向于仍按照原指令行事，而非被诱导。⁷⁹实验结果表明，将安全前端与结构化微调结合（StruQ）或与安全对齐结合（SecAlign），可大幅降低提示注入成功率，同时保持模型原有能力不下降^{80 81}。具体来说，在面对普通的（非优化）提示注入攻击时，StruQ模型将攻击成功率降至不到2%，SecAlign则完全防住（0% 成功率）^{82 81}；即使对于经过优化的强提示攻击，StruQ也显著降低了成功率，SecAlign更是将成功率压低到15%以下^{82 81}。这证明通过对抗训练，LLM的防御能力可以实质性提高。不过需要注意，SecAlign在强攻击下仍有少许漏网，这与前述Carlini等人的结论一致：**没有绝对安全的防御，只有逐步强化的模型**。未来的攻击会继续针对现有防御进行优化，因此对抗训练也需不断引入新出现的攻击样本迭代更新，形成持续的训练-对抗循环。

在模型加固之外，还需建立**对抗评估**机制来验证训练效果。例如构建专门的基准数据集来评测模型在各种攻击下的表现。Toyer等人（2023）已收集整理了一批提示注入攻击与防御的数据集，用于分析不同模型易受攻击的模式并评测防护手段⁸³。OpenAI也开源了Eval平台，允许开发者编写**对抗性单元测试**（例如让模型执行某项禁止操作，看其是否拒绝），将其纳入模型部署前的打分体系。随着攻防样本的积累，我们有望制定行业标准的安全评测集，让不同模型的对抗鲁棒性具有可比性和量化指标。

需要强调，对抗训练应谨慎拿捏平衡。如果过度惩罚模型顺从指令的倾向，可能导致**过度拒绝或性能下降**。Bianchi等人（2023）发现，过度的指令微调可能仍产生有害内容，他们提出加入安全数据来共同微调以折中安全与性能⁸⁴。Zhao等人（2023a）研究了在微调过程中模型如何“学会”或“遗忘”不安全示例，提出了过滤微调数据中不安全内容的ForgetFilter技术，在不显著牺牲性能前提下提升安全性⁸⁵。因此，对抗样本训练需要精细设计，避免把模型变“笨”或者过于保守。在实践中，往往结合**规则防御**（如在应用层增加输入输出过滤）与**模型优化**两方面手段，提供多层次的保障⁸⁶。

综上所述，对抗样本的生成与训练为提升LLM安全提供了有力工具。通过不断将最新的攻击套路纳入训练和评估，我们期望构建出在面对已知**Prompt攻击、输出操控、行为诱导**时依然可靠的模型。当然，这一过程需要持续投入，正如安全领域的箴言：“进攻总在进化，防守亦须如此”。

开源工具、业界实践与案例

开源工具方面，除了前述的Promptfoo和DeepTeam，社区还涌现了其他辅助LLM安全的项目。例如，LangChain社区推出了Rebuff工具用于检测和阻止提示注入攻击⁸⁷；微软等公司开源了指导性**安全基线**和检查清单，帮助开发者在构建LLM应用时遵循最佳实践。另一个值得关注的项目是OWASP组织的**“大语言模型应用安全Top10”**，其中详细列出了LLM应用面临的十大主要风险（如提示注入、数据泄露、模型偏见、身份验证漏洞等）并给出了缓解建议⁸⁸。OWASP还在GitHub上维护了针对LLM的安全测试用例库。这些开源资源为企业内部红队提供了参考基准，可以对照检查自家模型系统是否存在共性漏洞。

业界实践方面，各大科技公司纷纷建立AI红队或安全评审流程，将红队测试融入模型开发部署的生命周期中。例如OpenAI建立了专门的**Red Teaming网络**，在GPT-4等模型发布前邀请外部专家进行深入测试⁴⁴。这些专家的发现被纳入系统卡报告，许多高风险问题（如多模态提示注入、工具滥用路径等）在模型正式上线前得以及时缓解^{45 46}。微软也组建了AI红队，曾在内部模拟攻击Office Copilot、Bing Chat等系统，检查其访问企业数据时的边界保护。2023年，某大型金融公司在部署客服AI代理后，通过内部红队发现其存在提示注入导致的敏感数据泄露漏洞，及时避免了更大损失^{89 90}。事后分析表明，该AI因为信任了用户提供的隐藏指令而输出了受保护的信息，而传统安全控制（如网络隔离、防火墙）对此无能为力^{90 9}。此事件促使该公司加强了AI代理的**身份认证和权限管理**，为每个AI代理分配独立凭据并严格限定其数据访问范围，仿照对人类用户的零信任原则来管理AI身份^{91 92}。

在**攻防对抗工具**方面，许多企业开始将LLM安全测试集成到开发流程。例如有的团队在CI流水线加入Promptfoo/DeepTeam脚本，对每次模型更新运行一遍预定义的对抗测试用例。如果出现新的失败案例则阻断部署并通知开发修复。这类传统软件中的单元测试和渗透测试相结合的做法，使模型安全问题能够“Shift Left”（前移）至开发阶段解决。还有公司开发了**实时监控**系统，在AI代理运行时检查其输入输出是否有异常模式，一旦检测到类似提示注入的可疑模式就进行拦截或警报^{91 93}。例如Obsidian Security提供的AI安全代理就宣称可实时监测LLM行为，一旦模型尝试执行越权操作（如批量导出数据或调用敏感API）会立即阻断^{94 91}。另外，**法规遵循**也推动业界实践，如NIST的AI风险管理框架（AI RMF）和ISO 42001标准开始要求在AI系统中落实提示注入防护措施⁹⁵。这些标准督促企业建立完善的AI安全治理，包括定期红队测试、权限最小化配置以及事故响应预案等，从制度上保障LLM安全。

值得一提的实践案例是**Slack AI私聊泄露事件**和**Google Bard提示攻击**。安全研究者曾演示，通过在Slack某频道中发布特殊格式的信息，能诱导Slack的AI总结功能输出其他私人频道的内容（属于跨上下文的数据泄露）。Google的Bard也一度被发现可以通过提示让其泄露内部文档摘要。这些案例推动了厂商修复各自产品，将**系统指令与用户内容隔离**，例如对不同来源的上下文打标签、限制模型引用未经授权的内容等^{96 97}。总体而言，业界正逐步形成共识：**大模型系统的安全需要“深度防御”**（defense in depth），从模型训练、应用架构到运行监控各层面都要考虑安全。开发者不仅要在模型层面加强对抗鲁棒性，也要在应用层增加策略防护，如输入清理、输出审查、严格的身份权限管控和日志审计等^{91 92}。多家公司的经验表明，这种多层防御能将事故响应成本降低约60-70%，远胜于事后补救⁹⁸。

未来研究趋势

展望未来，随着攻击与防御持续演进，LLM红队测试和安全研究将出现以下趋势：

1. 更强的自适应攻防对抗：攻击者将越来越多地利用生成式AI来增强攻击。例如用一个“AI攻击代理”自动探索目标LLM的弱点，实时调整提示策略。这要求防御方也使用AI辅助来**生成更多样本的对抗攻击用于测试**，并在防御策略中预想攻击者可能的适应性行为。正如Nasr等人（2025）强调的，未来防御研究必须针对聪明且投入大量资源的攻击者设计，只有这样才能提出经得起考验的鲁棒防御^{65 66}。因此我们可能看到**生成-对抗-训练**循环加速：模型遇到新型攻击 -> 失败样本加入训练改进模型 -> 攻击者再找更隐蔽攻击 -> 如此往复，不断逼近模型安全极限。

2. 多模态和复杂环境下的安全：随着LLM接入视觉、语音、代码执行等能力，新攻击面层出不穷。例如**图像中的隐写提示**（在用户上传图片中隐藏指令让视觉LLM执行），**语音指令投毒**（用人耳难辨的对抗性音频指令攻击语音助手）等。这方面GPT-4o的红队结果已经显现端倪，未来需要开发**跨模态红队**方法来评估模型在图像、音频、视频输入下的安全性^{99 100}。另外，Agent系统变得更加自主和复杂（如AutoGPT类可以连续执行任务），其决策链可能出现新型漏洞，例如**任务规划阶段的攻击**（攻击者提供恶意中间反馈诱导Agent错误计划）等。研究人员需要跟进这些复杂场景，提出相应的攻击分析与防护方案。

3. 模型结构与推理过程安全：目前大多数防御仍是通过外部约束或后期微调来实现。未来或许需要从模型结构入手，提高其“天生”的安全属性。例如，有学者探讨在**模型架构中加入安全控制模块**，实时监测模型的注意力或中间生成是否被异常模式主导，从而在模型内部及时截断潜在的不良输出。这类类似于在模型推理过程中植入“旁路刹车”机制。还有研究方向是**可验证的安全LLM**，尝试给出理论保证：模型在什么条件下必不输出某类危险内容。这可能涉及可解释性、形式化验证等交叉领域，目前仍在起步阶段。

4. 供应链和数据源安全：企业内部部署的模型系统，其安全不止取决于模型本身，也包括所用第三方组件、插件和数据源的安全。未来红队可能会扩展到**模型供应链**，比如检查模型文件或权重是否被植入后门^{101 102}；验证向量数据库、检索系统是否存在被攻击者操纵的入口。尤其在RAG系统中，内部知识库的数据质量和真实性将直接影响模型输出可信度，因而需要开发**数据完整性审计**和**对抗性检索测试**的方法。例如模拟攻击者干预检索排序（通过巧妙构造查询或投毒语料使不相关文档优先检索）来看系统是否健壮^{103 104}。这些方向将丰富红队测试的范畴，从而覆盖LLM应用的全链条安全。

5. 人机协同防御：再强大的模型也难免有漏洞，因此“人类反馈+AI”的结合将是长期策略。当模型不确定如何安全响应时，及时引入人类审核（Human in the Loop）可防止事故扩大。未来研究可能探索如何检测模型何时陷入困境或可能被诱骗，从而**自动请求人类干预**。这需要模型具备不确定性评估能力，以及流程上允许人工快速介入处理异常请求。不少企业已经在高风险操作（如财务决策、代码执行）中引入二次确认，未来这一机制或更加智能化。

总的来说，保障企业级LLM和Agent系统的安全是一场**持久战**。攻击技术会不断翻新，促使防御措施同步演进。从近期研究和实践看，业界正在从被动补丁转向主动出击，通过红队测试查漏补缺、通过对抗训练增强模型免疫。可以预见，在**大型模型+工具**成为常态的未来数字业务中，红队安全研究将扮演更加举足轻重的角色，成为AI系统研发部署不可或缺的一环。只有这样，我们才能最大限度地发挥大模型的潜力，同时将其潜在风险控制可在可接受范围之内，为企业和用户保驾护航。

参考文献：本文内容参考了近期大量研究论文、技术博客和安全报告，包括Chowdhury等《大型语言模型攻击综述》¹⁰⁵、Nasr等《自适应攻击绕过LLM防御》⁶²、Chen等USENIX '25论文及相关项目报告^{106 80}，Obsidian Security博客^{94 91}、Promptfoo文档^{17 18}、Keysight安全公告^{26 27}等，在此一并致谢。各引用出处已在文中标明。

1 2 4 5 6 7 8 9 10 11 12 13 14 24 25 89 90 91 92 93 94 95 98 Prompt Injection

Attacks: The Most Common AI Exploit in 2025

<https://www.obsidiansecurity.com/blog/prompt-injection>

3 76 77 78 79 80 81 82 88 96 97 106 USENIX Sec'25 | LLM提示词注入攻击如何防？UC伯克利、Meta最新研究来了-腾讯云开发者社区-腾讯云

<https://cloud.tencent.com/developer/article/2496454>

15 16 35 36 41 42 68 69 70 71 72 73 74 75 83 84 85 86 101 102 105 Breaking Down the Defenses: A Comparative Survey of Attacks on Large Language Models

<https://arxiv.org/html/2403.04786v2>

17 18 19 20 21 22 23 33 34 37 38 39 40 53 54 55 103 104 How to red team RAG applications | Promptfoo

<https://www.promptfoo.dev/docs/red-team/rag/>

26 27 28 29 30 CVE-2024-8309: Prompt Injection in LangChain's GraphCypherQACHain Leads to Full Database Compromise

<https://www.keysight.com/blogs/en/tech/nwvs/2025/08/29/cve-2024-8309>

31 32 NVD - CVE-2025-46059

<https://nvd.nist.gov/vuln/detail/CVE-2025-46059>

43 44 45 46 47 48 49 50 51 52 60 61 67 99 100 OpenAI: Red Teaming GPT-4o, Operator, o3-mini, and Deep Research

<https://control-plane.io/case-studies/openai-red-teaming/>

56 57 58 59 GitHub - confident-ai/deepteam: DeepTeam is a framework to red team LLMs and LLM systems.

<https://github.com/confident-ai/deepteam>

62 63 64 65 66 攻击者后发制人：更强大的自适应攻击突破LLM越狱与提示注入防御 - 论文详情

<https://www.chatpaper.ai/zh/dashboard/paper/db499b2c-069b-4570-9532-fb6e520203bd>

87 Rebuff: Detecting Prompt Injection Attacks - LangChain Blog

<https://blog.langchain.com/rebuff/>